

MOT REMINDER SYSTEM

Complete App & Web System Documentation

PROJECT DOCUMENTATION

Application Type:	Mobile (React Native) & Web Customer Portal
Backend Services:	Node.js (Express), MongoDB (Mongoose)
Documentation Scope:	Objective, Role Permissions, Flowcharts, APIs, Models
System Status:	Production-ready MVP
Release Version:	v1.0.0
Generated Date:	August 13, 2026

1. Objectives & Business Scope

The MOT Reminder Management System is a specialized software solution designed for UK garages to automate Motor Operating Test (MOT) notifications and streamline booking conversions. By keeping track of customer vehicle expiry dates, the platform eliminates manual office administration and boosts business retention through timely, automated client communications.

Core Business Goals:

- **Maximize Booking Conversions:** Automating reminders at critical touchpoints reduces booking dropoffs and increases garage revenue.
- **Minimize Operational Overhead:** Manual tracking of spreadsheets and phone calling is replaced with background cron jobs and SMS/Email messaging.
- **Ensure Data Integrity:** Automates validation of vehicle ownership (Sold, Scrapped, Active) and registers new vehicles securely.
- **DVLA Integration:** Direct integration with the Driver and Vehicle Licensing Agency (DVLA) database fetches real-time vehicle details and expiry dates instantly, avoiding typing mistakes.

2. Architecture & Technology Stack

The platform is divided into three key technological layers: the mobile client for garage administrators/staff, the web-based self-service customer portal, and the backend cloud server.

1. Frontend Mobile Application (React Native):

Built on top of React Native using TypeScript, providing a cross-platform (iOS and Android) interface for garage admins and mechanics. The app features state-driven navigation, a dark/light design system, and full context data mapping via custom contexts.

2. Customer Self-Service Web Portal (Next.js/HTML & Tailwind):

A responsive, mobile-first web interface that requires zero authentication or logins. Customers access this secure portal through tokenized magic links sent directly via SMS, WhatsApp, or Email.

3. Backend Server & Database (Node.js + Express + MongoDB):

A high-performance RESTful API server. It features JWT-based role-based authentication, custom route security middleware, automated daily cron check runners, and database abstraction via Mongoose schemas on MongoDB.

3. System Roles & Access Matrix

The platform enforces role-based access control (RBAC) to segment administrative controls, operational tools, and user self-services.

Feature Area	Super Admin	Garage Staff	Customer Portal
Dashboard Metrics	Full Access	Full Access	No Access
Customer/Car Creation	Full Access	Full Access	Read-Only Details
DVLA Search Lookup	Full Access	Full Access	No Access
Alert Approvals Queue	Full Access	Full Access / Manage	No Access
Message Templates	Read & Write	No Access	No Access
Staff Acc Management	Create & Delete	No Access	No Access
System Audit Logs	Read-Only	No Access	No Access
Self-Service Actions	No Access	No Access	Full Portal Submissions

4. Step-by-Step Working Flows

A. Automated Reminder & Customer Booking Flow:

- Daily Cron Scanner executes at 8:00 AM on the backend database.
- The scanner computes: (MOT Expiry Date - Today's Date).
- If the difference is exactly 45, 30, or 7 days, it generates a message.
- An SMS/WhatsApp is sent to the customer containing a customized token link: `https://motapp.co.uk/update?token=base64_encoded_string`.
- The customer taps the link, loading their vehicle's detailed profile on the portal.
- Customer chooses 'Book MOT', selects a slot, and clicks Submit.
- An alert is instantly dispatched to the garage admin queue.
- Garage staff confirms the date/time slot. The status changes to Approved, audit logs are saved, and the customer is booked.

B. Customer Vehicle Sold / Scrapped Flow:

1. Customer opens the SMS link and realizes they no longer own the vehicle.
2. Customer clicks 'Mark Vehicle Sold' on the self-service web portal.
3. A 'SOLD' alert notification request is posted to the backend database.
4. The garage dashboard displays a pending alert badge count.
5. Admin opens the Alerts screen, verifies the customer's request, and clicks 'Approve'.
6. The backend changes the vehicle status in the database from 'Active' to 'Sold'.
7. Future automated cron checks for this vehicle registration are instantly terminated, stopping all reminders.

C. Customer Registers New Vehicle Flow:

1. Customer taps 'Add New Vehicle' on their secure web portal.
2. The customer enters the new car registration plate (VRN), Make, and Model.
3. Clicking submit sends a 'NEW_VEHICLE' alert request to the garage approval queue.
4. Admin logs into the system, reviews the alert details in the queue, and selects 'Approve'.
5. The backend automatically registers the new vehicle under the customer's ID.
6. A yearly reminder cron starts tracking this new plate, maintaining automatic communications.

D. Admin User/Staff Management Flow:

1. Admin logs into the Mobile app, clicks 'Settings' (Profile) tab.
2. Admin fills out the Staff Registry form: Full Name, Email, and Password.
3. On click, the app calls `/api/auth/create-staff`, hashing the password and creating the staff role.
4. Admin can tap 'View Staff List' to see all active mechanics or deletes a staff member if needed.

5. Complete REST API Endpoint Specification

All interactions between the Mobile App, Web Portal, and Backend Server are handled via JSON REST endpoints.

Method	Route Path	Role Required	Operation Description
POST	/api/auth/login	Public	User credentials authentication; returns JWT token.
POST	/api/auth/create-staff	Admin Only	Creates a new staff account under the garage.
GET	/api/auth/staff	Admin Only	Fetches list of active staff accounts.
DELETE	/api/auth/staff/:id	Admin Only	Deletes a staff account by ID.
GET	/api/customers	Admin, Staff	Retrieves complete customer listings.
GET	/api/customers/search	Admin, Staff	Advanced query filtering for search parameters.
GET	/api/customers/:id	Admin, Staff, Client	Retrieves single customer profile and vehicle lists.
POST	/api/customers	Admin, Staff	Saves a new customer record into database.
POST	/api/vehicles	Admin, Staff	Adds a new vehicle registration detail.
GET	/api/vehicles/dvla:vrn	Admin, Staff	Calls UK DVLA to lookup make/model by plate.
GET	/api/alerts	Admin, Staff	Fetches queue of pending customer actions.
PUT	/api/alerts/:id/approve	Admin, Staff	Approves alert (updates vehicle status or registers new car).
PUT	/api/alerts/:id/reject	Admin, Staff	Rejects portal requests with custom reason.
PUT	/api/alerts/:id/reschedule	Admin, Staff	Changes booking appointment date and time slots.
GET	/api/reminders/logs	Admin, Staff	Retrieves audit reports of sent message alerts.
PUT	/api/reminders/templates	Admin Only	Updates text templates for reminder cron dispatches.
POST	/api/reminders/trigger-cron	Admin, Staff	Runs background checking routine immediately.
GET	/api/response/portal	Public (Magic Link)	Decodes encrypted token and loads client info.
POST	/api/response/portal/action	Public (Magic Link)	Submits customer self-service choice.
GET	/api/dashboard/stats	Admin, Staff	Compiles statistics, metrics, and conversion counters.
GET	/api/reports/due-mots	Admin, Staff	Exports lists of vehicles expiring within range.

6. Database Model Schemas

The database utilizes MongoDB. Below are descriptions of key schemas defined in the backend codebase.

1. User Collection (Schema details for accounts):

- **username:** String, required, unique (Username of the employee).
- **email:** String, required, unique, lowercase (Email identifier).
- **password:** String, required (BCrypt hashed string).
- **role:** String, enum: ['admin', 'staff', 'customer'], default: 'customer'.

2. Customer Collection (Database record of clients):

- **firstName / lastName:** String, required (Customer's full name details).
- **email / mobile:** String, required, unique (Preferred contact addresses).
- **preferredMethod:** String, enum: ['SMS', 'Email', 'WhatsApp'], default: 'SMS'.
- **vehicles:** Array of ObjectIds, referencing the Vehicles collection.

3. Vehicle Collection (Client cars details):

- **customerId:** ObjectId, referencing the Customer collection.
- **registrationNumber:** String, required, unique, uppercase (UK Reg Plate).
- **make / model / year:** String / Number, required (Auto-filled via DVLA or input).
- **motExpiryDate:** Date, required (Tracking date for reminder schedule).
- **status:** String, enum: ['Active', 'Sold', 'Scrapped', 'Pending', 'Rejected'], default: 'Active'.

4. Alert Collection (Queue of portal notifications):

- **type:** String, enum: ['BOOKED', 'SOLD', 'NEW_VEHICLE'], required.
- **status:** String, enum: ['Pending', 'Approved', 'Rejected', 'Acknowledged'], default: 'Pending'.
- **customerName / customerId:** String / ObjectId (Associated client details).
- **registrationNumber / makeModel:** String (Subject vehicle identifiers).
- **date:** Date (Used for booking slot records).